

CHAPTER 4. Security of File Uploads and Implementing Secure PHP Practices

1. Introduction

File uploading is a common feature in web applications. Users may need to upload:

- Profile pictures
- Documents
- PDF files
- Assignment submissions
- IDs and certificates
- Reports
- ZIP files

However, **file uploads are also a major security risk** if they are not properly validated and handled.

An attacker may attempt to upload a malicious file such as a PHP script instead of a legitimate document. If the server executes the uploaded file, the attacker could potentially gain unauthorized access to the web application.

Therefore, developers should implement **secure file-upload practices in PHP**.

2. Common File Upload Security Threats

A. Malicious File Upload

An attacker may upload a PHP file disguised as an image or document.

Example:

malicious.php

If the server allows PHP files to be uploaded into an executable directory, the uploaded script could potentially be executed.

Security principle: Never trust the file extension supplied by the user.

B. Double Extension

An attacker might try to upload:

`profile.jpg.php`

The attacker may hope that the application recognizes it as an image while the server recognizes it as a PHP script.

Solution: Validate the actual file type and generate a new server-side filename.

C. MIME-Type Spoofing

A browser can send a MIME type such as:

`image/jpeg`

even when the file is not actually a JPEG image.

Therefore, this is **not sufficient**:

```
if ($_FILES['file']['type'] == 'image/jpeg') {  
    // Accept file  
}
```

The MIME type supplied by the client should not be trusted.

D. Oversized Files

An attacker may upload a very large file to consume:

- Disk space
- Memory
- CPU resources
- Network bandwidth

This can potentially cause a **Denial-of-Service (DoS)** condition.

A secure application should impose a file-size limit.

E. Path Traversal

Attackers may attempt to manipulate filenames using values such as:

```
../../important-file.txt
```

The goal is to make the application write files outside the intended upload directory.

Solution: Do not use the user's filename directly when storing files.

F. Malicious File Content

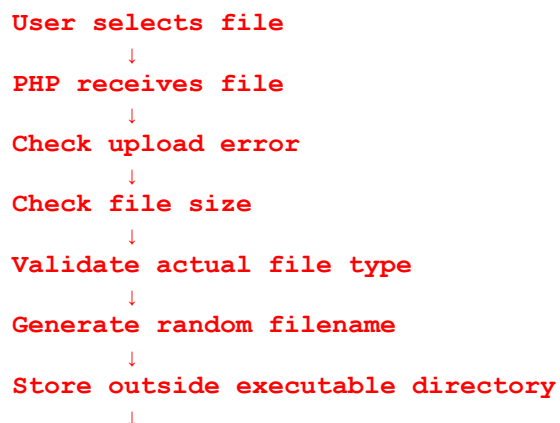
Even an image or PDF can potentially contain malicious content.

Therefore, developers should:

1. Validate the file type.
2. Limit file size.
3. Rename the file.
4. Store it safely.
5. Restrict executable files.

3. Secure PHP File Upload Practices

A secure upload process can follow this model:



```
Set secure permission\\\
↓
Save file information in database
```

4. Validate the Upload Error

PHP provides `$_FILES` for uploaded files.

Example:

```
if (!isset($_FILES['document'])) {
    die("No file uploaded.");
}

if ($_FILES['document']['error'] !== UPLOAD_ERR_OK) {
    die("File upload failed.");
}
```

This helps ensure that the upload process completed successfully.

5. Limit File Size

Example:

```
$maxSize = 5 * 1024 * 1024; // 5 MB

if ($_FILES['document']['size'] > $maxSize) {
    die("File is too large.");
}
```

The application will reject files larger than **5 MB**.

6. Validate File Extensions

Define an allowlist:

```
$allowedExtensions = [
    'jpg',
    'jpeg',
    'png',
    'pdf'
];

$extension = strtolower(
    pathinfo($_FILES['document']['name'], PATHINFO_EXTENSION)
);
```

```
if (!in_array($extension, $allowedExtensions, true)) {  
    die("File type is not allowed.");  
}
```

An **allowlist** is safer than trying to maintain a blacklist.

Avoid:

```
if ($extension != 'php') {  
    // Accept  
}
```

This is weak because there are many other potentially dangerous file types.

Instead, specify exactly what is allowed:

```
jpg  
jpeg  
png  
pdf
```

7. Validate the Actual MIME Type

For images, PHP provides `finfo_file()`.

```
$finfo = new finfo(FILEINFO_MIME_TYPE);  
  
$mimeType = $finfo->file($_FILES['document']['tmp_name']);  
  
$allowedMimeTypes = [  
    'image/jpeg',  
    'image/png',  
    'application/pdf'  
];  
  
if (!in_array($mimeType, $allowedMimeTypes, true)) {  
    die("Invalid file type.");  
}
```

This is stronger than trusting:

```
$_FILES['document']['type']
```

because that value originates from the client.

8. Generate a Random Filename

Never rely on the original filename.

Instead of:

```
move_uploaded_file(
    $_FILES['document']['tmp_name'],
    "uploads/" . $_FILES['document']['name']
);
```

use a generated filename:

```
$newFilename = bin2hex(random_bytes(16)) . "." . $extension;
```

Example:

```
83a91c4f2a8e7d91f3c1b62e8a4d9912.pdf
```

This prevents problems involving:

- Filename collisions
- Special characters
- Path traversal
- Predictable filenames

9. Use Carefully

If you need to display the original filename, you can use:

```
$originalName = basename($_FILES['document']['name']);
```

However, `basename()` **should not be considered a complete security solution.**

It is still better to generate a random server-side filename.

10. Store Uploaded Files Safely

A safer architecture is:

```
public_html/
|
|— index.php
|— login.php
```

```
| uploads/
```

But an even safer approach is to store sensitive uploads **outside the publicly accessible web directory**.

For example:

```
/home/account/
├── public_html/
│   ├── index.php
│   └── login.php
└── private_uploads/
    ├── document1.pdf
    └── document2.pdf
```

The PHP application can then authenticate the user before allowing access to the file.

11. Prevent Script Execution

If files must be stored inside a web-accessible upload directory, prevent script execution.

For Apache, an `.htaccess` file inside the upload directory can be used to disable PHP execution.

For example:

```
<FilesMatch "\.(php|php[0-9]?|phtml)$">
    Require all denied
</FilesMatch>
```

This is an additional defense layer.

Important: The preferred solution is still to keep untrusted uploads outside the web root whenever possible.

12. Secure File Upload Example in PHP

HTML Form

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
```

```
<label>Select PDF or Image:</label>

<input
    type="file"
    name="document"
    accept=".jpg,.jpeg,.png,.pdf"
    required
>

<button type="submit">Upload</button>

</form>
```

The important part is:

```
enctype="multipart/form-data"
```

Without it, file uploads will not work correctly.

13. Secure upload.php

```
<?php

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    exit('Invalid request.');
```

```
}

if (!isset($_FILES['document'])) {
    exit('No file selected.');
```

```
}

$file = $_FILES['document'];

// 1. Check upload error
if ($file['error'] !== UPLOAD_ERR_OK) {
    exit('Upload failed.');
```

```
}

// 2. Maximum file size: 5 MB
$maxSize = 5 * 1024 * 1024;

if ($file['size'] > $maxSize) {
    exit('File is too large.');
```

```
}

// 3. Allowed MIME types
$allowedTypes = [
    'image/jpeg' => 'jpg',
    'image/png'  => 'png',
    'application/pdf' => 'pdf'
];

// 4. Detect actual MIME type
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mimeType = $finfo->file($file['tmp_name']);
```

```
if (!isset($allowedTypes[$mimeType])) {
    exit('File type is not allowed.');
```



```
}

// 5. Generate safe extension
$extension = $allowedTypes[$mimeType];

// 6. Generate random filename
$newFilename = bin2hex(random_bytes(16)) . '.' . $extension;

// 7. Upload directory
$uploadDirectory = __DIR__ . '/uploads/';

// 8. Create directory if necessary
if (!is_dir($uploadDirectory)) {
    mkdir($uploadDirectory, 0755, true);
}

// 9. Final path
$destination = $uploadDirectory . $newFilename;

// 10. Move uploaded file
if (!move_uploaded_file($file['tmp_name'], $destination)) {
    exit('Unable to save uploaded file.');
```



```
}

echo "File uploaded successfully.";

?>
```

14. Why This Code Is More Secure

The code implements several security controls:

Security Practice	Implementation
Upload error validation	UPLOAD_ERR_OK
File size restriction	5 MB
MIME validation	finfo
Extension allowlist	JPG, PNG, PDF
Random filename	random_bytes()
Original filename not trusted	Server-generated name
Safe file movement	move_uploaded_file()
Directory control	Dedicated upload folder

15. Additional Security Practices

File upload security should not stop at validation.

Authentication

Only authorized users should be allowed to upload files.

```
session_start();

if (!isset($_SESSION['user_id'])) {
    exit('Unauthorized access.');
```

Authorization

A logged-in user should not automatically have permission to upload everything.

For example:

```
Student
  ↓
Can upload assignment

Teacher
  ↓
Can upload course materials

Administrator
  ↓
Can manage uploaded files
```

Use role-based access control where appropriate.

CSRF Protection

Forms that change server data should use CSRF protection.

Example concept:

```
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

Then include the token in the form and verify it during submission.

Database Security

If storing file information in MySQL, use **PDO prepared statements**.

Example:

```
$stmt = $pdo->prepare("
    INSERT INTO files
    (original_name, stored_name, mime_type, file_size)
    VALUES (?, ?, ?, ?)
");

$stmt->execute([
    $originalName,
    $newFilename,
    $mimeType,
    $file['size']
]);
```

Never construct SQL using raw user input.

16. Recommended Database Structure

A simple table could be:

```
CREATE TABLE files (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    original_name VARCHAR(255) NOT NULL,
    stored_name VARCHAR(255) NOT NULL,
    mime_type VARCHAR(100) NOT NULL,
    file_size INT NOT NULL,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Notice the difference between:

original_name

and

stored_name

For example:

Original:
My_Activity.pdf

Stored:
a81f3d7c9e214b0a8f12c4d9e1a55c21.pdf

The user can see the original name while the server uses a safer generated filename.

17. Security Principle: Never Trust User Input

One of the most important principles in secure PHP programming is:

Never trust data coming from the client.

This includes:

```
$_GET
$_POST
$_FILES
$_COOKIE
$_SERVER
```

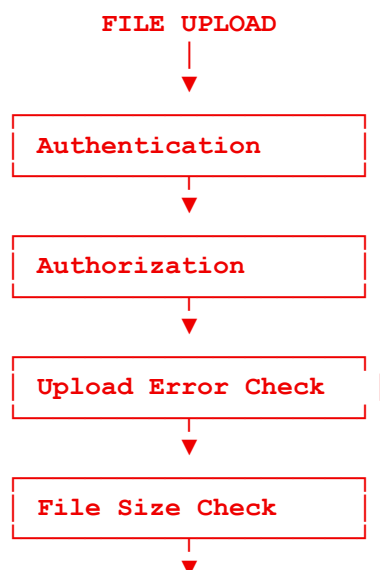
For file uploads, this means you should **not automatically trust**:

```
$_FILES['document']['name']
$_FILES['document']['type']
$_FILES['document']['size']
```

Instead, validate everything on the server.

18. Defense-in-Depth Approach

A secure file-upload system should use multiple layers:





The idea is that **no single security control should be relied upon.**